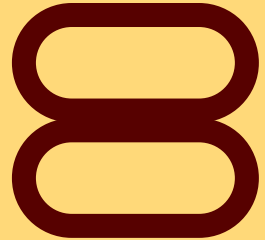
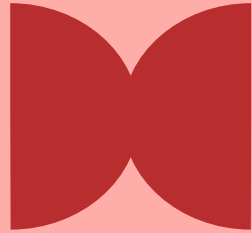


RDMA Based KV Store



Venkatesh
Axe
Parth
Dibri

github.com/venkateshdb/KV_RDMA

Problem:

TCP KVs Hit a Bottleneck

✦ Traditional KV Store (TCP)

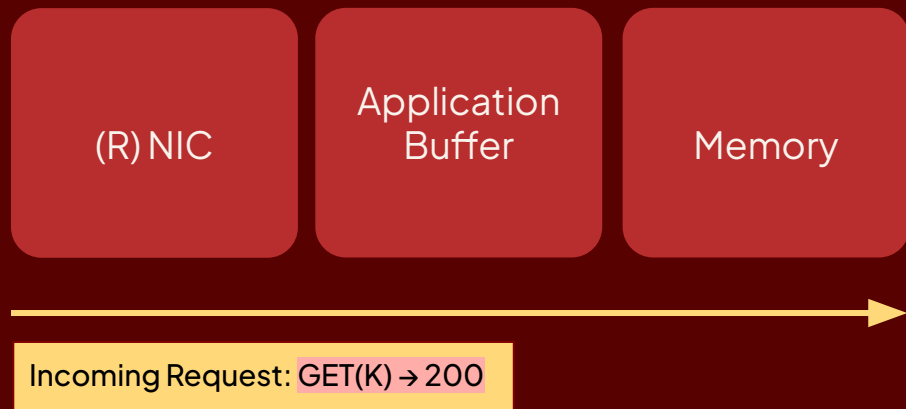


Incoming Request: GET(K) → 200

The CPU and kernel must process every request but RDMA eliminates this path.

What even is RDMA?

✦ Proposed KV Store (w RDMA)

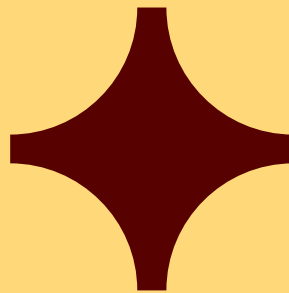


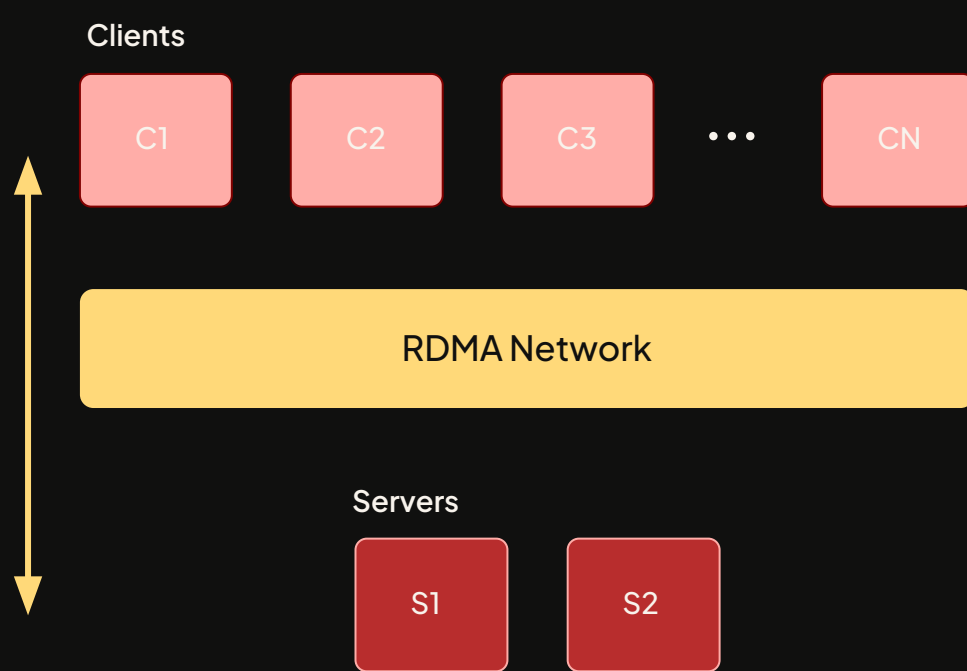
RDMA allows us to bypass the syscalls to read/write memory on a remote server.

How it works:

- Server exposes a registered memory region (MR).
- Client receives its address + rkey.
- Client issues one-sided ops via a Queue Pair (QP).
- NIC directly DMA's data into server memory.

System Design



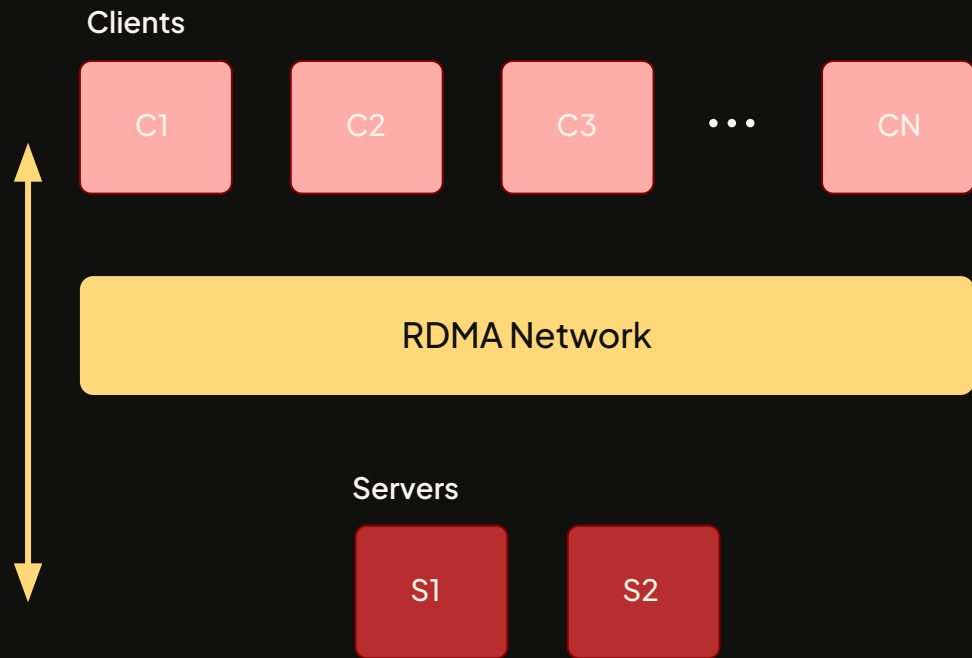


Client Details

1. Local Buffer
2. QP + CQ
3. RDMA Operations

Server Details

1. 1GB Memory Region
2. Registered for Remote Access
3. Zero-Copy Operations



Features:

- KV Operations from PA1.
- Direct memory access without CPU.
- Modulo-based sharding across servers.
- Batch operations for improved throughput.

Key Questions

1. Does RDMA meaningfully reduce latency?



2. Does it scale better than TCP?



3. Does CPU utilization drop?



4. Can we preserve correctness?

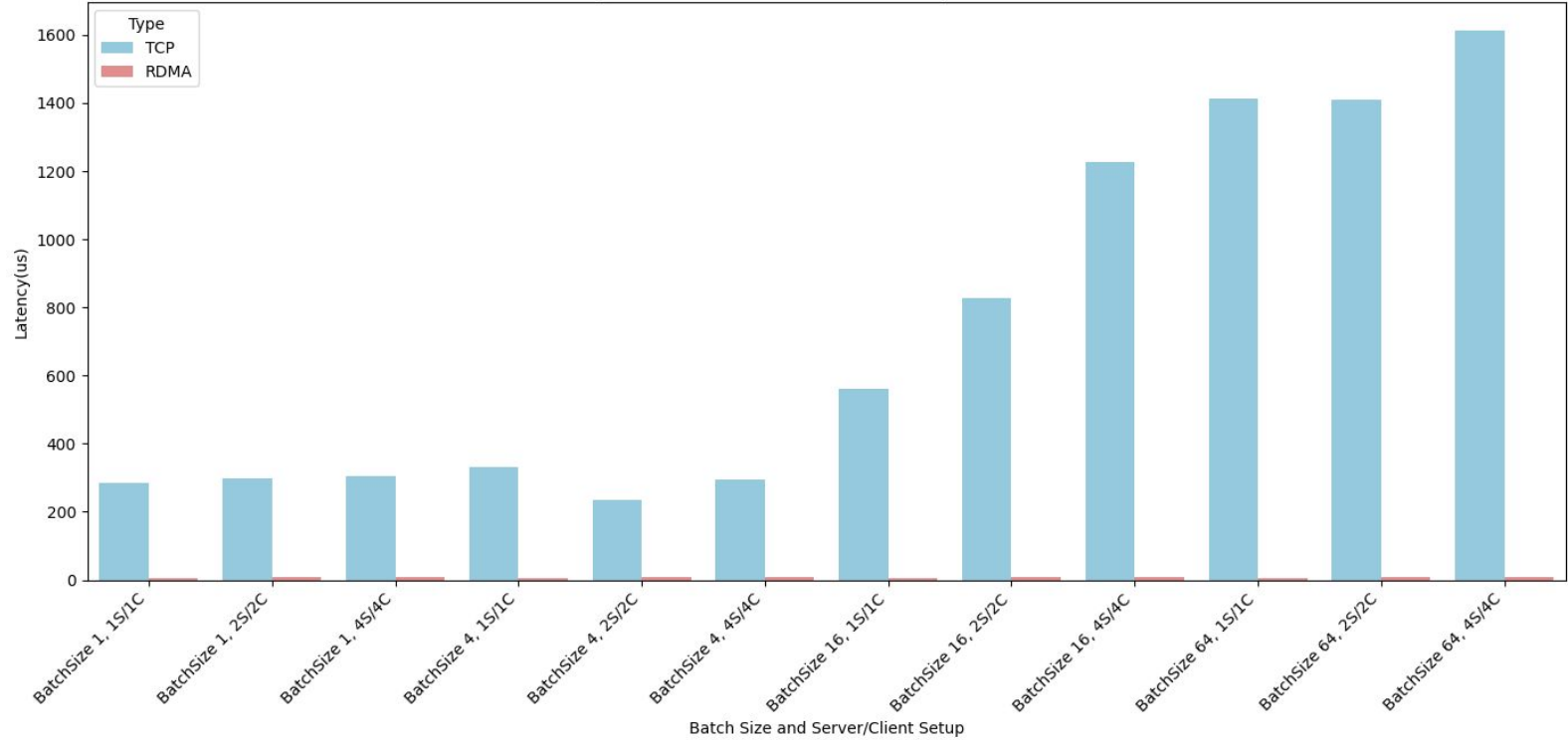


RDMA vs TCP

Batch size	TCP latency (μ s)	RDMA latency (μ s)	Latency improvement
1	298.26	9.47	$\approx 31\times$ lower
4	235.26	8.42	$\approx 28\times$ lower
16	828.04	8.02	$\approx 100\times$ lower
64	1,410.04	7.82	$\approx 180\times$ lower

In every case, RDMA stays in single-digit microseconds while TCP is in the hundreds to thousands of microseconds

Latency across different Server/Client Setups and Batch Sizes



Key Questions

1. Does RDMA meaningfully reduce latency?



2. Does it scale better than TCP?



3. Does CPU utilization drop?



4. Can we preserve correctness?

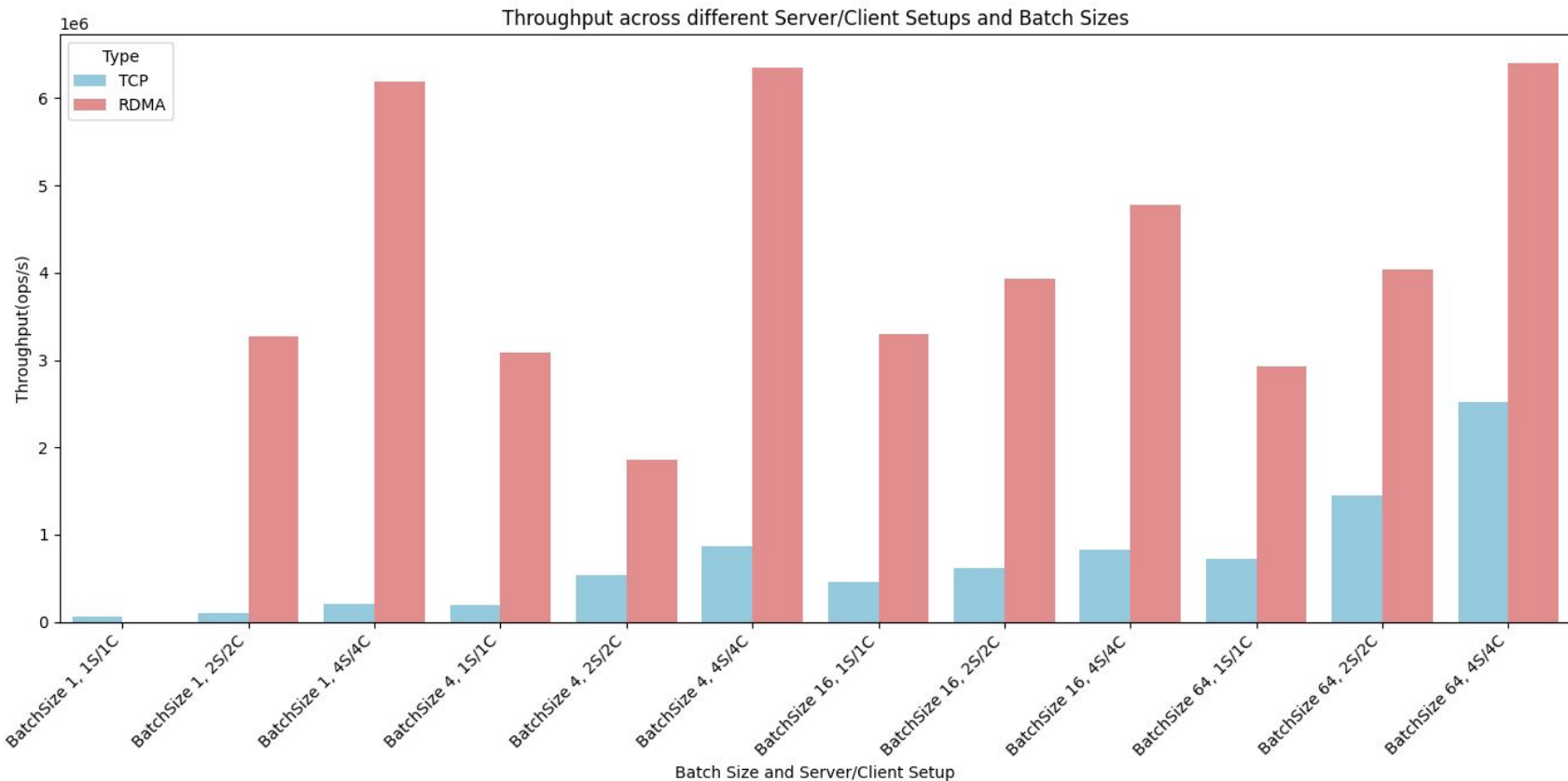


RDMA vs TCP

Servers/Clients (batch 1, 16 workers)	TCP throughput (ops/s)	RDMA throughput (ops/s)	Scale-up TCP (2→4)	Scale-up RDMA (2→4)
2 / 2	106,413	3,269,680	—	—
4 / 4	208,589	6,190,190	$\approx 2.0\times$	$\approx 1.9\times$

Both scale roughly $2\times$ when doubling servers from 2→4, but RDMA runs at $\sim 30\times$ the absolute throughput at each point.





Key Questions

1. Does RDMA meaningfully reduce latency?



2. Does it scale better than TCP?



3. Does CPU utilization drop?



4. Can we preserve correctness?



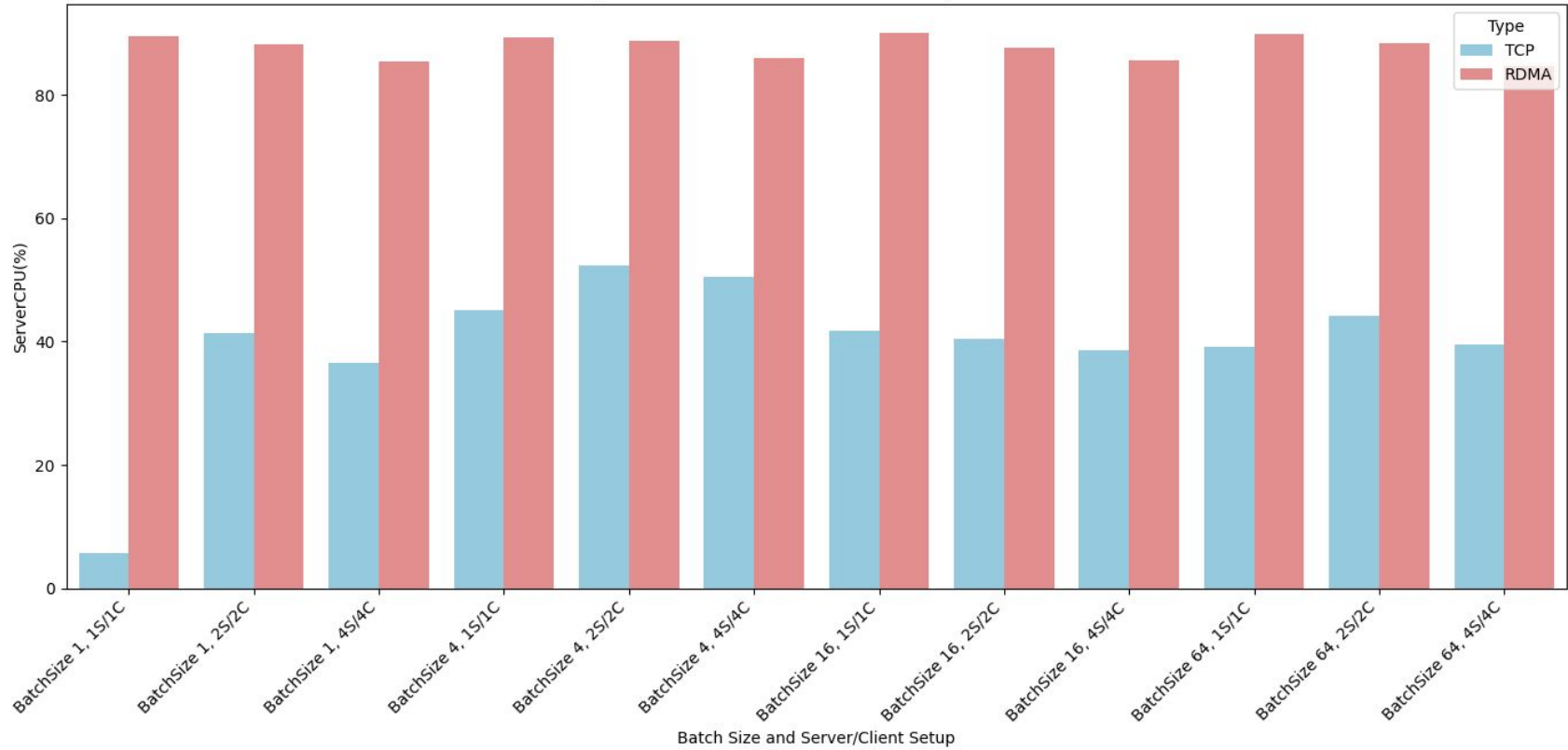
RDMA vs TCP

Batch size	TCP CPU%	RDMA CPU%	Latency TCP → RDMA	Throughput TCP → RDMA
1	41.42	88.25	298 μ s → 9.5 μ s	0.11M → 3.27M ops/s
4	52.37	88.86	235 μ s → 8.4 μ s	0.54M → 1.86M ops/s
16	40.47	87.73	828 μ s → 8.0 μ s	0.61M → 3.93M ops/s
64	44.23	88.52	1,410 μ s → 7.8 μ s	1.44M → 4.04M ops/s

CPU utilization does **not** drop with RDMA since we're polling the CQ



CPU Usage across different Server/Client Setups and Batch Sizes



Key Questions

1. Does RDMA meaningfully reduce latency?



2. Does it scale better than TCP?



3. Does CPU utilization drop?



4. Can we preserve correctness?



Problems

1. Regular RDMA Write/Read does not provide linearizability.
2. Local CPU write can break linearizable semantics.

Planned Solution

Compare-and-swap (CAS). Still need to figure out the details of how coordinate with the CPU, since `ibv_wr_atomic_cmp_swp()` does not provide atomic guarantees on CPU-initiated writes.



Takeaways

1. RDMA meaningfully reduces latency in our KV store.



2. Our RDMA approach scales better than with just TCP.



3. We notice CPU utilization does not drop.



4. More work to be done to ensure preservation of correctness.



Team Contributions

Venkatesh: Worked on RDMA setup, memory registration, and server-side data region configuration.

Axe: Helped set up QP creation and server-side structure for one-sided operations.

Parth: Implemented parts of the client-side RDMA GET/PUT pipeline and began integrating YCSB-B.

Dibri: Set up the baseline TCP benchmarking environment and debugging the early RDMA test cases.